

Trực quan hóa dữ liệu với Matplotlib

**Từ biểu đồ cơ bản đến quy trình phân tích dữ liệu
có thể tái lập**

Bài giảng Data Science

Cập nhật: 24/08/2026

Một biểu đồ tốt giúp trả lời câu hỏi, không chỉ “làm đẹp” dữ liệu

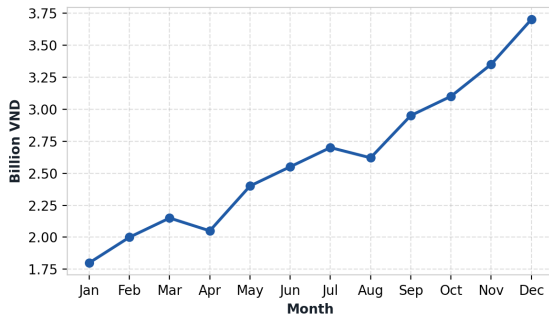
Bảng số liệu cho biết

12 giá trị doanh thu theo tháng, chính xác đến từng con số.

Biểu đồ cho thấy

Xu hướng tăng, giai đoạn chững lại và mùa cao điểm cuối năm.

Monthly Revenue Trend



Thông điệp: trực quan hóa nén dữ liệu thành mẫu hình mà con người nhận ra nhanh.

Mục tiêu học tập

Sau bài học, người học có thể:

- giải thích mô hình **Figure-Axes** và chọn đúng API;
- tạo line, bar, scatter, histogram và pie chart;
- tùy biến tiêu đề, nhãn, ticks, legend, màu, marker và grid;
- bố trí nhiều biểu đồ bằng subplots;
- tạo heatmap, contour, 3D và animation ở mức nhập môn;
- tích hợp Matplotlib với NumPy, Pandas và Seaborn;
- xuất hình chất lượng cao và tránh các lỗi trình bày gây hiểu sai.

Lộ trình bài học đi từ câu hỏi đến sản phẩm trực quan

- 1 Matplotlib là gì và nằm ở đâu trong hệ sinh thái Python?
- 2 Figure, Axes và hai phong cách viết mã.
- 3 Năm loại biểu đồ nền tảng.
- 4 Tùy biến, chú thích và bố cục nhiều biểu đồ.
- 5 Heatmap, 3D, contour, animation và tương tác.
- 6 Pandas/Seaborn, lưu hình và quy trình Data Science.
- 7 Quiz, bài tập có hướng dẫn và bài tập tự làm.

1. Nền tảng Matplotlib

1. Nền tảng Matplotlib

Hiểu đúng thư viện trước khi viết mã

Matplotlib là lớp nền trực quan hóa của hệ sinh thái Python

- Thư viện mã nguồn mở cho hình **tĩnh**, **động** và **tương tác**.
- Làm việc tự nhiên với NumPy ndarray và Pandas DataFrame.
- Kiểm soát chi tiết trục, nhãn, legend, màu, font và layout.
- Xuất PNG, PDF, SVG và nhiều định dạng khác.
- Có thể nhúng vào Tkinter, Qt, GTK hoặc wxPython.

Vai trò thực tế

Khám phá: tìm lỗi, ngoại lệ, xu hướng.

Giải thích: truyền đạt kết quả phân tích.

Sản xuất: tạo hình cho báo cáo, dashboard và ứng dụng.

Cài đặt và kiểm tra môi trường chỉ cần vài lệnh

```
# Terminal
python -m pip install matplotlib

# Conda
conda install matplotlib
```

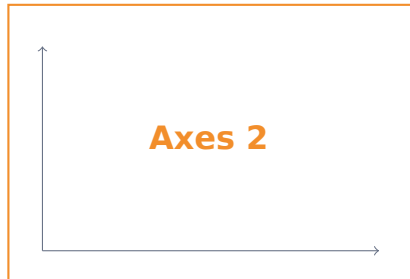
```
1 import matplotlib
2 import matplotlib.pyplot as plt
3
4 print(matplotlib.__version__)
```

Trong Jupyter Notebook

Với backend inline mặc định, hình thường xuất hiện ngay dưới cell. `plt.show()` vẫn nên dùng để mã rõ ràng và chạy ổn định ngoài notebook.

Một Figure chứa một hoặc nhiều Axes

Figure: toàn bộ “tờ giấy”



Axes là vùng vẽ có hệ trục; Axis là từng trục x/y. Không nên dùng lẫn hai thuật ngữ.

Pyplot nhanh; Object-Oriented rõ ràng và dễ mở rộng

Pyplot interface

```
1 plt.plot(x, y)
2 plt.title("Revenue")
3 plt.xlabel("Month")
4 plt.show()
```

Phù hợp: thử nhanh, một biểu đồ đơn giản.

Object-Oriented API

```
1 fig, ax = plt.subplots()
2 ax.plot(x, y)
3 ax.set(title="Revenue",
4         xlabel="Month")
5 plt.show()
```

Phù hợp: notebook lớn, subplot, hàm tái sử dụng.

Khuyến nghị trong bài này: ưu tiên Object-Oriented API.

Quy trình vẽ ổn định gồm bốn bước

- 1 Chuẩn bị dữ liệu.
- 2 Tạo fig, ax.
- 3 Vẽ và tùy biến trên ax.
- 4 Hiển thị hoặc lưu fig.

```
1 x = [1, 2, 3, 4]
2 y = [12, 18, 17, 25]
3
4 fig, ax = plt.subplots()
5 ax.plot(x, y)
6 ax.set(title="Sales")
7 fig.tight_layout()
8 plt.show()
```

2. Năm biểu đồ nền tảng

2. Năm biểu đồ nền tảng

Chọn biểu đồ theo câu hỏi phân tích

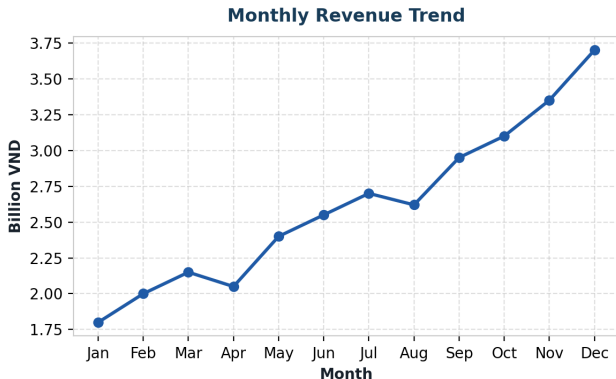
Loại câu hỏi quyết định loại biểu đồ

Câu hỏi	Biểu đồ	Ví dụ
Thay đổi theo thời gian?	Line	Doanh thu theo tháng
So sánh các nhóm?	Bar	Doanh số theo khu vực
Hai biến liên hệ thế nào?	Scatter	Quảng cáo và doanh số
Một biến phân bố ra sao?	Histogram/boxplot	Thời gian giao hàng
Cơ cấu tổng thể?	Pie/stacked bar	Tỷ trọng kênh bán
Nhiều giá trị trong ma trận?	Heatmap	Tương quan, dữ liệu thiếu

Không chọn biểu đồ chỉ vì “trông đẹp”; hãy bắt đầu từ câu hỏi.

Line chart nhấn mạnh xu hướng và thứ tự thời gian

```
1 months = range(1, 13)
2 revenue = [1.8, 2.0, 2.15,
3           2.05, 2.4, 2.55, 2.7,
4           2.62, 2.95, 3.1, 3.35, 3.7]
5
6 fig, ax = plt.subplots()
7 ax.plot(months, revenue,
8         marker="o")
9 ax.set(xlabel="Month",
10        ylabel="Billion VND")
11 plt.show()
```



Khi đọc line chart, hãy tìm bốn loại tín hiệu

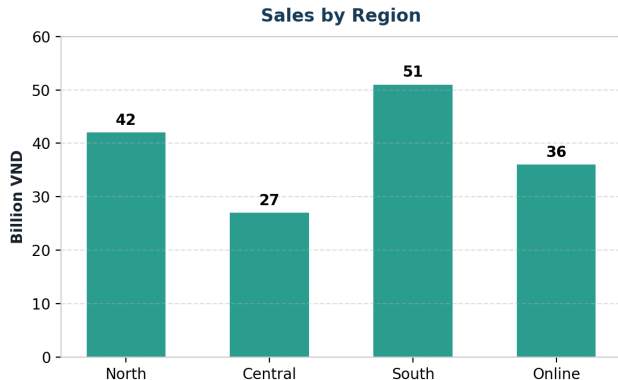
- **Mức độ:** giá trị điển hình cao hay thấp?
- **Xu hướng:** tăng, giảm hay đi ngang?
- **Mùa vụ:** mẫu hình có lặp lại theo chu kỳ?
- **Bất thường:** điểm nào lệch khỏi hành vi chung?

Cẩn thận

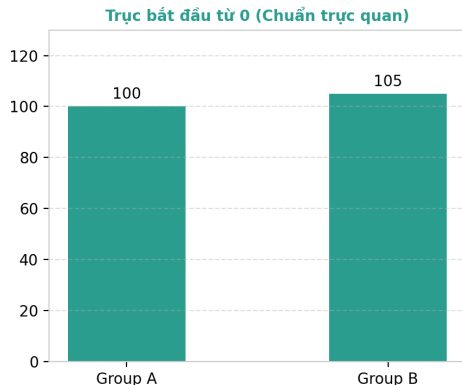
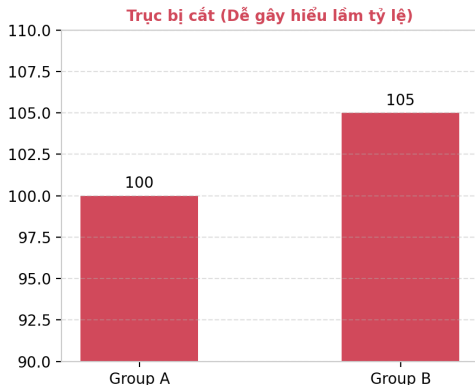
Không nối các nhóm không có thứ tự tự nhiên. Đường nối ngầm khẳng định tính liên tục hoặc trình tự.

Bar chart giúp so sánh độ lớn giữa các nhóm

```
1 regions = ["North", "Central",  
2           "South", "Online"]  
3 sales = [42, 27, 51, 36]  
4  
5 fig, ax = plt.subplots()  
6 bars = ax.bar(regions, sales)  
7 ax.bar_label(bars)  
8 ax.set(ylabel="Billion VND")  
9 plt.show()
```



Bar chart nên bắt đầu trục giá trị từ 0



- Chiều dài cột mã hóa độ lớn; cắt trục có thể phóng đại chênh lệch.
- Với line chart, trục không nhất thiết bắt đầu từ 0 nếu mục tiêu là xem biến động.

Bar ngang hữu ích khi nhãn dài hoặc cần xếp hạng

```
1 order = np.argsort(sales)
2
3 ax.barh(
4     np.array(regions)[order],
5     np.array(sales)[order]
6 )
7 ax.set_xlabel("Sales")
```

Quy tắc thực hành

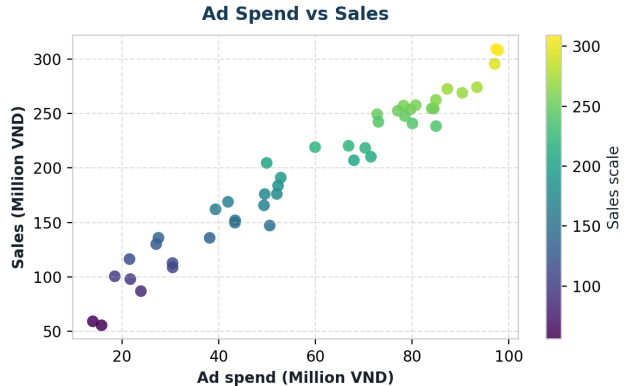
- Sắp xếp khi muốn nhấn mạnh thứ hạng.
- Giữ thứ tự thời gian hoặc thứ tự nghiệp vụ khi nó có ý nghĩa.
- Hạn chế dùng quá nhiều màu nếu màu không mã hóa thông tin.

Scatter plot làm lộ ra quan hệ, cụm và ngoại lệ

```

1 fig, ax = plt.subplots()
2 sc = ax.scatter(
3     ad_spend, sales,
4     c=sales, cmap="viridis",
5     s=55, alpha=.8
6 )
7 ax.set(xlabel="Ad spend",
8       ylabel="Sales")
9 fig.colorbar(sc, ax=ax)

```



Scatter plot có thể mã hóa thêm biến nhưng dễ quá tải

- Vị trí x, y: hai biến số chính.
- Màu c: nhóm hoặc biến số thứ ba.
- Kích thước s: quy mô, doanh thu hoặc tần suất.
- Độ trong suốt alpha: giảm hiện tượng chồng điểm.

Nguyên tắc

Chỉ thêm một kênh thị giác khi người đọc có thể giải thích nó và legend/colorbar làm rõ ý nghĩa.

Histogram mô tả hình dạng của một phân phối

```
1 fig, ax = plt.subplots()
2 ax.hist(delivery_days,
3         bins=18,
4         edgecolor="white")
5 ax.axvline(
6     delivery_days.mean(),
7     linestyle="--"
8 )
9 ax.set(xlabel="Days",
10        ylabel="Orders")
```



Số bins thay đổi câu chuyện mà histogram kể

Bins quá ít

Che mất nhiều đỉnh, khoảng trống và ngoại lệ.

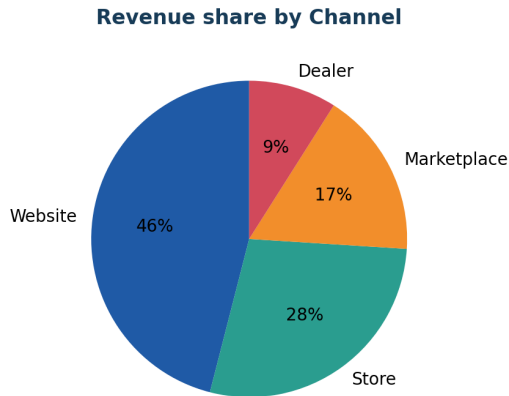
Bins quá nhiều

Khuếch đại nhiễu ngẫu nhiên và làm hình khó đọc.

- Thử nhiều giá trị hợp lý thay vì tin một histogram duy nhất.
- Có thể dùng `bins="auto"` làm điểm khởi đầu.
- So sánh các nhóm cần dùng chung cạnh bins.

Pie chart chỉ phù hợp với cơ cấu đơn giản

```
1 sizes = [46, 28, 17, 9]
2 labels = ["Website", "Store",
3           "Marketplace", "Dealer"]
4
5 ax.pie(sizes, labels=labels,
6        autopct="%1.0f%%",
7        startangle=90)
8 ax.set_title("Revenue share")
```



Pie chart mất hiệu quả khi có nhiều lát hoặc chênh lệch nhỏ

Có thể dùng khi

- tổng đúng bằng 100%;
- có ít nhóm;
- các tỷ trọng khác biệt rõ.

Nên đổi sang bar khi

- cần so sánh chính xác;
- có nhiều nhóm;
- có số âm hoặc tổng không rõ.

Con người so sánh chiều dài tốt hơn góc và diện tích.

3. Tùy biến để biểu đồ tự giải thích

3. Tùy biến để biểu đồ tự giải thích

Nhãn, màu và chú thích phải phục vụ thông điệp

Tiêu đề và nhãn trục là phần của phân tích

```
1 fig, ax = plt.subplots(figsize=(8, 4.5))
2 ax.plot(quarters, actual, marker="o", label="Actual")
3 ax.plot(quarters, target, marker="s", linestyle="--", label="Plan")
4
5 ax.set_title("Actual sales exceeded plan in Q4", loc="left")
6 ax.set_xlabel("Quarter")
7 ax.set_ylabel("Sales (billion VND)")
8 ax.legend()
```

Tiêu đề tốt

Nêu kết luận chính khi biểu đồ dùng để trình bày; nêu biến và phạm vi khi biểu đồ dùng để khám phá.

Màu, marker và line style phân biệt chuỗi dữ liệu

```
1 ax.plot(x, y1,  
2         color="#1F5AA6",  
3         marker="o",  
4         linewidth=2.5)  
5  
6 ax.plot(x, y2,  
7         color="#F28E2B",  
8         marker="s",  
9         linestyle="--")
```

- Màu: nhấn mạnh hoặc mã hóa nhóm.
- Marker: hỗ trợ bản in đen trắng.
- Line style: phân biệt thực tế/kế hoạch.
- alpha: xử lý chồng lấp.

Legend chỉ hữu ích khi người đọc cần giải mã

```
1 ax.plot(x, actual, label="Actual")
2 ax.plot(x, target, label="Plan")
3 ax.legend(loc="upper left", ncols=2, frameon=False)
```

- Gán `label=` ngay khi vẽ mỗi series.
- Đặt legend ở vùng trống, tránh che dữ liệu.
- Nếu chỉ có một series và tiêu đề/nhãn đã rõ, có thể bỏ legend.
- Gắn nhãn trực tiếp vào đường đôi khi tốt hơn legend.

Grid và ticks hỗ trợ đọc số nhưng không nên lấn át dữ liệu

```
1 ax.grid(axis="y", alpha=.25)
2 ax.set_xticks(range(4),
3               ["Q1", "Q2", "Q3", "Q4",
4               "])
5 ax.tick_params(axis="x",
6               rotation=0,
7               labelsize=11)
```

Tối giản có chủ đích

- Grid mờ và thường chỉ theo một trục.
- Không hiển thị quá nhiều tick labels.
- Ghi rõ đơn vị trong nhãn trục.
- Dùng formatter cho tiền tệ, phần trăm, ngày tháng.

Annotation biến một điểm dữ liệu thành thông tin hành động

```

1 ax.annotate(
2     "Above plan",
3     xy=(3, actual[3]),
4     xytext=(2.1, 124),
5     arrowprops={
6         "arrowstyle": "->"
7     }
8 )

```



Figure size và layout quyết định khả năng đọc

```
1 fig, ax = plt.subplots(figsize=(10, 5), constrained_layout=True)
2 # ... plotting commands ...
3
4 # Alternative for an existing figure
5 fig.tight_layout()
```

- `figsize=(width, height)` dùng đơn vị inch.
- `constrained_layout=True` xử lý khoảng cách tự động ngay khi tạo Figure.
- `tight_layout()` hữu ích cho bố cục đơn giản.
- Luôn kiểm tra hình ở kích thước thực tế của báo cáo/slide.

Style sheet tạo tính nhất quán với một lệnh

```
1 print(plt.style.available)
2
3 plt.style.use("seaborn-v0_8-whitegrid")
4
5 with plt.style.context("ggplot"):
6     fig, ax = plt.subplots()
7     ax.plot(x, y)
```

Cho dự án dài

Định nghĩa chuẩn màu, font, kích thước và grid bằng rcParams hoặc file .mplstyle; tránh tùy biến lại từng biểu đồ.

4. Nhiều biểu đồ và đồ thị nâng cao

4. Nhiều biểu đồ và đồ thị nâng cao

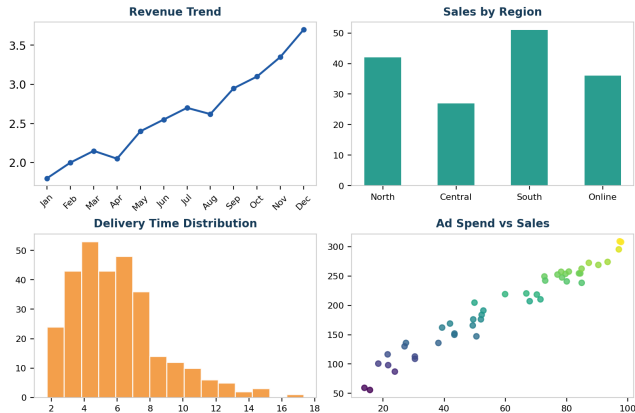
Mở rộng từ một Axes sang cấu trúc phân tích

Subplots cho phép so sánh nhiều góc nhìn trong cùng Figure

```

1 fig, axes = plt.subplots(
2     2, 2,
3     figsize=(10, 6),
4     constrained_layout=True
5 )
6
7 axes[0, 0].plot(month, revenue)
8 axes[0, 1].bar(region, sales)
9 axes[1, 0].hist(delivery)
10 axes[1, 1].scatter(ads, sales)

```



Chia sẻ trục giúp so sánh công bằng giữa các panels

```
1 fig, axes = plt.subplots(2, 1, sharex=True, sharey=True)
2
3 for ax, region in zip(axes, ["North", "South"]):
4     part = df[df["region"] == region]
5     ax.plot(part["month"], part["sales"])
6     ax.set_title(region)
```

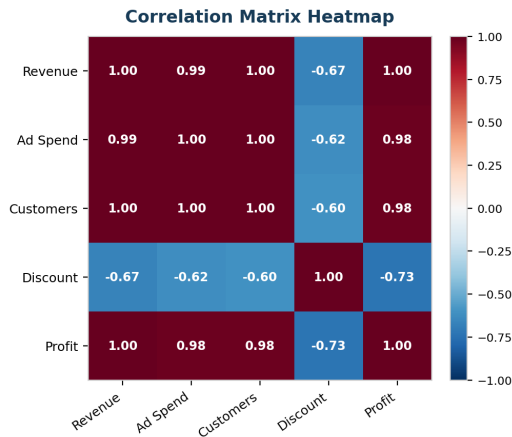
- Dùng chung scale khi mục tiêu là so sánh mức độ.
- Không ép chung scale nếu các đại lượng khác đơn vị hoặc khác bản chất.
- Giữ thứ tự panels nhất quán với câu chuyện phân tích.

Heatmap giúp quét nhanh ma trận tương quan

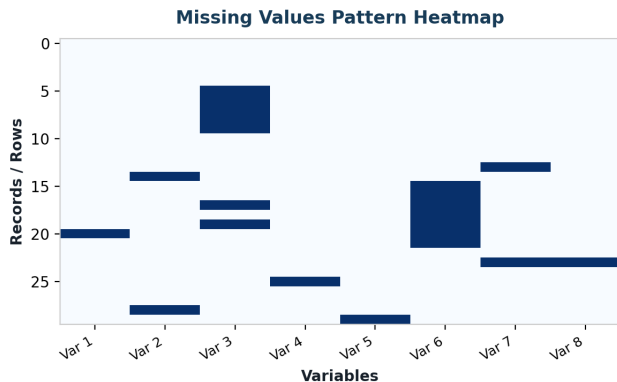
```

1 corr = df.corr(numeric_only=
2     True)
3
4 fig, ax = plt.subplots()
5 im = ax.imshow(
6     corr,
7     cmap="RdBu_r",
8     vmin=-1, vmax=1
9 )
10 fig.colorbar(im, ax=ax)

```



Heatmap hỗ trợ phát hiện tương quan và cấu trúc thiếu dữ liệu



- Cột nhiều ô thiếu: cần nhắc bỏ biến hoặc tìm nguồn thay thế.
- Hàng thiếu theo cụm: kiểm tra quy trình thu thập.
- Mẫu thiếu có cấu trúc: không nên mặc định là ngẫu nhiên.

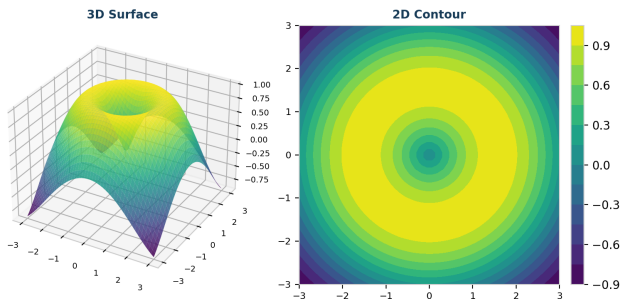
Heatmap giúp quét nhanh; không thay thế biểu đồ cần đọc giá trị chính xác.

Contour thường dễ đọc hơn surface 3D

```

1 X, Y = np.meshgrid(x, y)
2 Z = f(X, Y)
3
4 fig = plt.figure()
5 ax3d = fig.add_subplot(
6     121, projection="3d")
7 ax3d.plot_surface(X, Y, Z,
8                   cmap="viridis"
9                   )
10 ax2d = fig.add_subplot(122)
11 ax2d.contourf(X, Y, Z)

```



3D chỉ nên dùng khi chiều thứ ba thật sự quan trọng

Dùng 3D cho

- bề mặt toán học;
- địa hình;
- mô hình không gian;
- tương tác xoay góc nhìn.

Tránh 3D cho

- bar/pie trang trí;
- so sánh chính xác;
- hình tĩnh bị che khuất;
- “làm biểu đồ ẩn tượng”.

Animation mô tả trạng thái thay đổi theo thời gian

```
1 from matplotlib.animation import FuncAnimation
2
3 fig, ax = plt.subplots()
4 line, = ax.plot([], [])
5
6 def update(frame):
7     line.set_data(x[:frame], y[:frame])
8     return line,
9
10 ani = FuncAnimation(fig, update, frames=len(x), interval=80)
11 ani.save("trend.gif", writer="pillow")
```

Animation phù hợp khi chuyển động là thông tin; không nên thay một biểu đồ tĩnh rõ ràng bằng hiệu ứng.

Widgets biến Figure thành công cụ khám phá nhỏ

- `matplotlib.widgets.Slider`: thay đổi ngưỡng hoặc tham số.
- `Button`: chạy lại phép tính hoặc đổi chế độ.
- `CheckButtons`: bật/tắt series.
- `RangeSlider`: lọc khoảng giá trị.

Giới hạn

Matplotlib có tương tác, nhưng dashboard web nhiều người dùng thường phù hợp hơn với Plotly, Bokeh, Streamlit hoặc Dash.

5. Pandas, Seaborn và xuất bản

5. Pandas, Seaborn và xuất bản

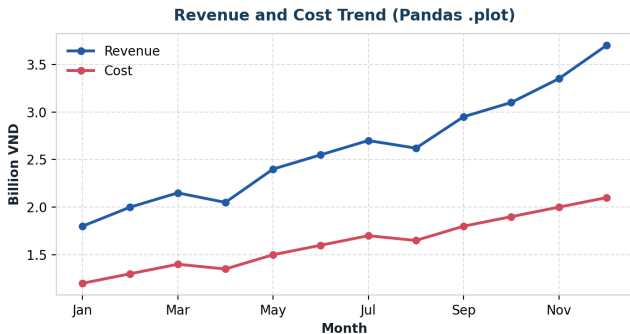
Từ phân tích nhanh đến hình dùng trong báo cáo

Pandas cung cấp API ngắn gọn nhưng vẫn trả về Axes

```

1 ax = df.plot(
2     y=["revenue", "cost"],
3     marker="o",
4     figsize=(8, 4)
5 )
6
7 ax.set_title("Revenue and cost")
8 ax.set_ylabel("Billion VND")
9 ax.legend(frameon=False)

```



Matplotlib và Seaborn bổ sung cho nhau

	Matplotlib	Seaborn
Mức trừu tượng	Thấp hơn, kiểm soát chi tiết	Cao hơn, thiên về thống kê
Dữ liệu	Mảng, chuỗi, DataFrame	DataFrame dạng tidy
Thế mạnh	Tùy biến, layout, xuất bản	Phân phối, nhóm, ước lượng
Kết quả	Figure/Axes	Vẫn dựa trên Matplotlib

Cách làm phổ biến: Seaborn vẽ nhanh, Matplotlib hoàn thiện tiêu đề, trục và bố cục.

Kết hợp Seaborn và Matplotlib không làm mất quyền kiểm soát

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 fig, ax = plt.subplots(figsize=(8, 4))
5 sns.boxplot(data=df, x="segment", y="order_value", ax=ax)
6 ax.set(title="Order value by customer segment",
7       xlabel="Customer segment", ylabel="Order value (VND)")
8 fig.tight_layout()
```

Ghi nhớ

Truyền rõ ax=ax giúp kiểm soát Figure và tích hợp biểu đồ vào subplot.

savefig xuất hình cho slide, web và bài báo

```
1 fig.savefig("revenue.png", dpi=300, bbox_inches="tight")
2 fig.savefig("revenue.pdf", bbox_inches="tight")
3 fig.savefig("revenue.svg", bbox_inches="tight")
```

PNG/JPG

Ảnh raster; phù hợp web, slide.

PDF/SVG

Vector; nét khi phóng to, phù hợp bài báo.

DPI

Quan trọng với raster; 150–300 thường đủ.

Lưu trước `plt.show()` để tránh khác biệt giữa các backend.

Một PDF có thể chứa nhiều Figure

```
1 from matplotlib.backends.backend_pdf import PdfPages
2
3 with PdfPages("report_figures.pdf") as pdf:
4     for region in regions:
5         fig, ax = plt.subplots()
6         plot_region(ax, df, region)
7         pdf.savefig(fig, bbox_inches="tight")
8         plt.close(fig)
```

Thực hành tốt

Đóng Figure trong vòng lặp bằng `plt.close(fig)` để tránh tăng bộ nhớ khi tạo hàng trăm hình.

Các toolkit mở rộng Matplotlib theo loại dữ liệu

- **mplot3d**: 3D tích hợp trong Matplotlib.
- **Seaborn**: biểu đồ thống kê cấp cao.
- **GeoPandas**: bản đồ từ GeoDataFrame, dùng Matplotlib làm backend.
- **Cartopy**: phép chiếu và dữ liệu địa lý chuyên sâu.
- **mplfinance**: biểu đồ tài chính.

Nguyên tắc chọn công cụ

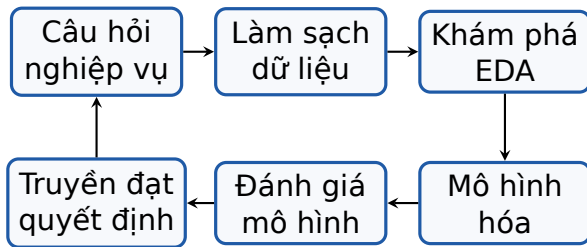
Dùng lớp trừu tượng cao để làm nhanh; quay về Figure/Axes khi cần kiểm soát bố cục và xuất bản.

6. Matplotlib trong quy trình Data Science

6. Matplotlib trong quy trình Data Science

Biểu đồ là công cụ kiểm tra giả định và truyền đạt kết quả

Trực quan hóa xuất hiện xuyên suốt vòng đời phân tích



EDA không phải “bước trang trí”; nó hỗ trợ phát hiện lỗi, chọn biến, biến đổi dữ liệu và hình thành giả thuyết.

Mục tiêu học máy định hướng cách trực quan hóa

Mục tiêu	Cách trực quan hóa hữu ích
Dự báo số	Đặt target ở trục y; scatter với predictor; histogram/boxplot để xem biến đổi
Phân loại	Boxplot predictor theo class; scatter tô màu theo class; confusion matrix
Chuỗi thời gian	Line chart ở nhiều mức tổng hợp; zoom vào giai đoạn bất thường
Không giám sát	Scatter matrix; heatmap tương quan; biểu đồ profile của cụm
Chất lượng dữ liệu	Histogram, boxplot, missing-value heatmap, kiểm tra giá trị bất hợp lệ

Sáu câu hỏi kiểm tra trước khi công bố biểu đồ

- 1 Biểu đồ trả lời câu hỏi nào?
- 2 Kiểu biểu đồ có phù hợp kiểu dữ liệu?
- 3 Trục, đơn vị và phạm vi có trung thực?
- 4 Màu có mang ý nghĩa hay chỉ trang trí?
- 5 Người đọc có thấy thông điệp trong 5-10 giây?
- 6 Mã và dữ liệu có đủ để tái lập hình?

Các lỗi phổ biến làm giảm độ tin cậy

Lỗi thiết kế

- trục bị cắt gây phóng đại;
- quá nhiều màu/legend;
- font và nhãn quá nhỏ;
- 3D che khuất dữ liệu;
- pie chart có quá nhiều lát.

Lỗi kỹ thuật

- dùng stateful API trong code phức tạp;
- quên đóng Figure trong vòng lặp;
- lưu ảnh DPI thấp;
- layout cắt nhãn;
- không cố định seed cho dữ liệu mô phỏng.

Đóng gói biểu đồ thành hàm giúp tái sử dụng và kiểm thử

```
1 def plot_monthly_sales(ax, data, title):
2     ax.plot(data["month"], data["sales"], marker="o")
3     ax.set(title=title, xlabel="Month", ylabel="Sales")
4     ax.grid(axis="y", alpha=.25)
5     return ax
6
7 fig, ax = plt.subplots(figsize=(8, 4))
8 plot_monthly_sales(ax, df, "Monthly sales")
9 fig.savefig("monthly_sales.svg", bbox_inches="tight")
```

Hàm nhận ax để ghép vào subplot và không phụ thuộc trạng thái toàn cục.

7. Củng cố và thực hành

7. Củng cố và thực hành

Kiểm tra hiểu biết rồi áp dụng vào dữ liệu kinh doanh

Quiz 1: Chọn biểu đồ phù hợp

Bạn cần so sánh phân phối giá trị đơn hàng giữa bốn phân khúc khách hàng. Biểu đồ nào phù hợp nhất?

- A. Line chart
- B. Side-by-side boxplot
- C. Pie chart
- D. 3D surface

Quiz 1: Chọn biểu đồ phù hợp

Bạn cần so sánh phân phối giá trị đơn hàng giữa bốn phân khúc khách hàng. Biểu đồ nào phù hợp nhất?

- ☐ A. Line chart
- ☐ B. Side-by-side boxplot
- ☐ C. Pie chart
- ☐ D. 3D surface

Đáp án: B

Boxplot cho thấy median, quartiles, độ phân tán và ngoại lệ của từng phân khúc.

Quiz 2: Figure và Axes

Trong lệnh `fig, ax = plt.subplots()`, phát biểu nào đúng?

- ☐ A. `fig` là trục x; `ax` là trục y.
- ☐ B. `fig` là toàn bộ canvas; `ax` là vùng vẽ.
- ☐ C. `fig` chỉ dùng để lưu PNG.
- ☐ D. `ax` chỉ dùng khi có nhiều subplot.

Quiz 2: Figure và Axes

Trong lệnh `fig, ax = plt.subplots()`, phát biểu nào đúng?

- ☐ A. `fig` là trục x; `ax` là trục y.
- ☒ B. `fig` là toàn bộ canvas; `ax` là vùng vẽ.
- ☐ C. `fig` chỉ dùng để lưu PNG.
- ☐ D. `ax` chỉ dùng khi có nhiều subplot.

Đáp án: B

Một Figure có thể chứa một hoặc nhiều Axes; Axes vẫn được dùng ngay cả khi chỉ có một biểu đồ.

Quiz 3: Phát hiện vấn đề trình bày

Một bar chart so sánh doanh số 98 và 100, nhưng trục y bắt đầu từ 97. Vấn đề chính là gì?

Quiz 3: Phát hiện vấn đề trình bày

Một bar chart so sánh doanh số 98 và 100, nhưng trục y bắt đầu từ 97. Vấn đề chính là gì?

Đáp án

Chiều dài cột bị diễn giải từ mốc 0, nên trục cắt làm chênh lệch 2% trông rất lớn. Nếu cần soi biến động nhỏ, dùng dot plot hoặc line chart kèm giá trị và giải thích phạm vi trục.

Bài tập mẫu: phân tích doanh thu theo tháng

Yêu cầu: vẽ line chart, thêm marker, đường trung bình và chú thích tháng cao nhất.

```
1 months = np.arange(1, 13)
2 revenue = np.array([18, 20, 22, 21, 24, 26, 27, 26, 30, 31, 34, 37])
3
4 fig, ax = plt.subplots(figsize=(9, 4.5))
5 ax.plot(months, revenue, marker="o")
6 ax.axhline(revenue.mean(), linestyle="--", color="gray", label="Mean")
7 i = revenue.argmax()
8 ax.annotate("Peak", xy=(months[i], revenue[i]), xytext=(9, 38),
9             arrowprops={"arrowstyle": "->"})
10 ax.set(xlabel="Month", ylabel="Revenue", title="Monthly revenue")
11 ax.legend(); fig.tight_layout()
```

Bài thực hành 1: Dashboard bán hàng 2x2

Từ DataFrame có các cột date, region, channel, revenue, cost, orders:

- 1 Tính doanh thu theo tháng và vẽ line chart.
- 2 Tính doanh thu theo vùng và vẽ bar chart đã sắp xếp.
- 3 Vẽ histogram giá trị đơn hàng trung bình.
- 4 Vẽ scatter giữa chi phí và doanh thu, tô màu theo kênh.
- 5 Ghép bốn biểu đồ vào một Figure 2x2, thêm tiêu đề chung.
- 6 Lưu kết quả thành PNG 300 DPI và PDF.

Gợi ý: groupby, resample, plt.subplots, fig.suptitle.

Bài thực hành 2: Chẩn đoán dữ liệu giao hàng

Với dữ liệu gồm `distance`, `weight`, `promised_days`, `actual_days`, `carrier`:

- 1 Tạo biến `delay = actual_days - promised_days`.
- 2 Dùng histogram để xem phân phối `delay`.
- 3 Dùng boxplot để so sánh `delay` giữa các hãng vận chuyển.
- 4 Dùng scatter để khảo sát `distance` và `actual_days`.
- 5 Chú thích ba đơn hàng trễ nhất.
- 6 Viết 3 nhận xét nghiệp vụ dựa trên biểu đồ.

Bài tập tự làm: sửa một biểu đồ gây hiểu sai

Tìm hoặc tự tạo một biểu đồ có ít nhất ba vấn đề:

- lựa chọn loại biểu đồ chưa phù hợp;
- trục hoặc đơn vị không rõ;
- màu/legend dư thừa;
- thiếu tiêu đề mang thông điệp;
- nhãn bị chồng hoặc quá nhỏ.

Nộp: (1) hình ban đầu, (2) hình đã sửa, (3) đoạn giải thích 150–200 từ, (4) mã Python có thể chạy lại.

Tóm tắt: từ câu hỏi đến quyết định

- **Figure-Axes** là mô hình tư duy cốt lõi; ưu tiên Object-Oriented API.
- Line, bar, scatter và histogram giải quyết phần lớn nhu cầu EDA cơ bản.
- Tùy biến chỉ có giá trị khi làm rõ dữ liệu hoặc thông điệp.
- Subplots, heatmap, 3D và animation phục vụ những cấu trúc câu hỏi cụ thể.
- Pandas/Seaborn giúp làm nhanh; Matplotlib giữ quyền kiểm soát cuối cùng.
- Xuất bản cần kiểm tra trực, đơn vị, layout, định dạng và khả năng tái lập.

Tài liệu tham khảo và học tiếp

-  Matplotlib Development Team. *Matplotlib Documentation*.
<https://matplotlib.org/stable/>
-  GeeksforGeeks. *Matplotlib Tutorial*, cập nhật 24/02/2026.
<https://www.geeksforgeeks.org/python-matplotlib-an-overview/>
-  G. Shmueli, P. C. Bruce, A. V. Deokar, N. R. Patel (2023).
Machine Learning for Business Analytics, Chương 3: Data Visualization.
Wiley.
-  S. Few (2012). *Show Me the Numbers*, 2nd ed. Analytics Press.